

Software Engineering Challenge

Operations Under Pressure

One-day full-stack engineering assessment

At Newtonite, engineers are expected to do more than implement specifications. We expect you to understand a problem, make reasonable assumptions, identify risks, design a coherent solution, and take ownership of the resulting system.

This exercise intentionally leaves several decisions unspecified. That is part of the assessment.

Candidate Brief

Newtonite Engineering

The Situation

A rapidly growing company currently coordinates operational work through a mixture of chat messages, spreadsheets, email, and direct conversations. This worked when the company had a small team. It no longer works with several hundred employees.

Every day, teams receive requests that require investigation or action. A request might be a customer issue, an engineering problem, a payment requiring investigation, a production incident, a compliance request, or an operational task requiring approval.

These requests often involve several people. Some are urgent. Some require approval before they can move forward. Some change ownership several times. Sometimes two people begin working on the same issue without realizing it. Sometimes one person changes a request while another person is still viewing or editing an older version.

Important requests occasionally disappear inside long message threads. Management has no reliable way to understand what work is happening, who owns it, what requires immediate attention, what changed, whether something has been forgotten, or why a particular decision was made.

The company wants to replace this workflow with an internal web application. You have been asked to design and build the first usable version.

Your Goal

Build a system that allows teams to coordinate and manage operational work reliably.

We intentionally have not specified exactly what the application should look like or how every workflow should behave. Part of this challenge is determining what a good solution should be. Make reasonable product and engineering decisions based on the situation above, and document important assumptions where appropriate.

What the System Should Enable

At a minimum, users should be able to create and manage operational work items. A work item should be capable of representing something that requires investigation, action, or resolution.

A user should be able to understand:

- what the item is and why it exists;
- its current state and importance;
- who is responsible for it;
- what has happened previously;
- what requires attention next.

Different users may interact with the same work item at approximately the same time. Your system should behave sensibly in these situations.

Teams, Identity, and Access

The company contains multiple teams. A user may belong to one or more teams and may have different responsibilities within them. Not every user should be able to perform every action.

Your solution should contain a meaningful authorization model. Authorization must be enforced by the system itself and should not rely only on hiding controls in the user interface. You may decide what roles or permission model make sense for your solution.

Collaboration and History

Users need to collaborate around work items and understand how an item has evolved over time. If responsibility changes, priority changes, or the item progresses through a workflow, other users should be able to understand what happened. Important actions should not silently disappear.

Concurrent Usage

Assume the application is being used by many employees simultaneously. Consider situations such as:

- two users attempting to take responsibility for the same work;
- one user viewing information that another user has just changed;
- multiple updates reaching the system within a short period;
- a user repeating an action because they are unsure whether the first request succeeded.

Your implementation should handle at least some of these situations deliberately rather than relying entirely on ideal user behavior.

User Experience

The application should provide a useful overview of ongoing work. A user should be able to quickly determine what requires attention and locate relevant work without manually browsing through everything in the system.

The application should remain usable as the amount of stored work grows. We care significantly more about a coherent and responsive experience than decorative UI.

System Behaviour

Some operations do not necessarily need to happen synchronously. Actions such as notifications, processing, enrichment, or other secondary work may happen after the primary action has completed.

You may introduce asynchronous processing if you believe it improves your architecture. If you do so, consider what happens when processing fails, executes more than once, or is delayed.

Expected Scale

You do not need to build infrastructure for millions of users. However, design the application as if the company expects continued growth. Assume approximately:

- thousands of registered users;
- hundreds to a few thousand simultaneous users;
- many teams;
- tens of thousands of active work items;
- a large and continually growing history of previous activity.

Your solution should not depend on loading the entire dataset into the browser or processing everything in application memory.

Engineering Expectations

We are not prescribing your technology choices. Choose technologies that allow you to deliver a reliable solution within the available time. We are interested in the reasoning behind those decisions.

Your submission should demonstrate meaningful consideration of:

- data modelling;
- API design;
- frontend state management;
- authorization;
- concurrent operations;
- error handling;
- data consistency;
- search and filtering;
- application performance;
- maintainability.

You are not expected to solve every possible distributed-systems problem. We would rather see a smaller system with well-considered behaviour than a large system containing many incomplete features.

Critical Behaviour

Your solution must demonstrate at least three situations where correctness requires more than simple CRUD operations.

Examples may include, but are not limited to:

- simultaneous actions by multiple users;
- preventing accidental duplicate operations;
- handling stale information;
- enforcing workflow rules;
- performing reliable asynchronous processing;
- handling authorization at the resource level;

- reconciling optimistic frontend state with server decisions.

You may choose which problems are most important for your design. Be prepared to explain them.

Engineering Decisions

Include a short document named `ENGINEERING_DECISIONS.md` (or equivalent) explaining approximately five important decisions you made while building the application.

These may include why you structured the data in a certain way, how you handle concurrent updates, what you chose to make synchronous versus asynchronous, how authorization works, or what you intentionally chose not to build.

We are interested in trade-offs. There is rarely one correct architectural answer.

Testing

You are not expected to achieve an arbitrary test-coverage percentage. Instead, identify behaviours that would be particularly dangerous if they were incorrect and write automated tests covering at least some of those behaviours. Your testing strategy should reflect the risks in your own architecture.

AI and Coding Tools

You may use coding assistants, AI agents, documentation, search engines, and other development tools. There is no penalty for doing so.

You are responsible for every architectural decision and every behaviour contained in your submission. During the review, you may be asked to explain or modify any part of your system.

Submission

Your submission should contain:

- working source code;
- clear instructions for running the application;
- any required setup instructions;
- your engineering decisions document;
- automated tests for important behaviour;
- a brief description of known limitations.

Architecture diagrams or additional documentation are welcome if they help explain your solution.

Time

You will have one working day to build your solution. Prioritize carefully.

We do not expect a production-complete enterprise platform. We expect a thoughtful engineering solution that demonstrates what you consider important when building reliable software.

Final Discussion

At the end of the exercise, you will demonstrate your application and discuss your implementation. Be prepared to explain:

- how your architecture works;
- what happens when things fail;
- what assumptions you made;
- which parts of the system you consider most important;
- what you would change if the application grew significantly;
- what you would work on next if you had another week.

The quality of your engineering reasoning is as important as the number of features completed.